OPERATING SYSTEMS AND SYSTEM PROGRAMMING

PRACTICAL – WEEK 3

Name: S Svara Haasini
Roll No: 2520090170

Task 1: Process Creation and State Observation Using fork()

Question:

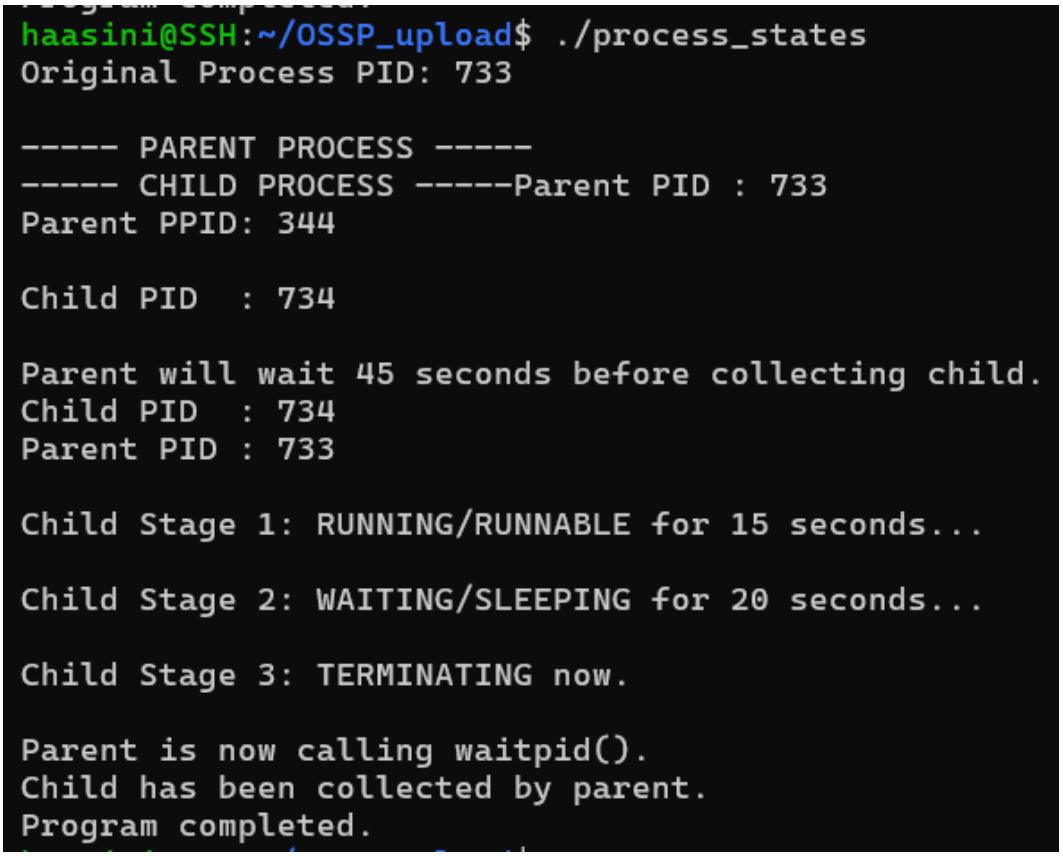
Develop a C program using fork() that creates a parent and child process. Display the Process ID (PID), Parent Process ID (PPID), and process states at different stages of execution.

Source Code

```
 GNU nano 7.2
#include <stdio.h>
#include <stdlib.h>
#include <unistd.h>
#include <sys/types.h>
#include <sys/wait.h>
#include <time.h>

int main(void)
{
    pid_t pid;

    /*
     * Disable output buffering so that every message
     * appears immediately on the terminal.
     */
    setbuf(stdout, NULL);

    printf("Original Process PID: %d\n\n", getpid());

    /*
     * fork() creates a new child process.
     */
    pid = fork();

    if (pid < 0)
    {
        perror("fork failed");
        return 1;
    }

    else if (pid == 0)
    {
        /*
         * CHILD PROCESS
         */
        printf("----- CHILD PROCESS -----\n");
        printf("Child PID  : %d\n", getpid());
        printf("Parent PID : %d\n\n", getppid());

        /*
         * STAGE 1:
         * Keep the CPU busy for 15 seconds.
         *
         * Linux will normally show this process
         * as R = Running/Runnable.
         */
        printf("Child Stage 1: RUNNING/RUNNABLE for 15 seconds...\n");

        time_t start = time(NULL);
        volatile unsigned long long counter = 0;

        while (time(NULL) - start < 15)
```

```c
        while (time(NULL) - start < 15)
        {
            counter++;
        }

        /*
         * STAGE 2:
         * sleep() makes the process wait.
         *
         * Linux normally shows this as:
         * S = Interruptible sleep
         */
        printf("\nChild Stage 2: WAITING/SLEEPING for 20 seconds...\n");

        sleep(20);

        /*
         * STAGE 3:
         * The child finishes.
         */
        printf("\nChild Stage 3: TERMINATING now.\n");

        _exit(0);
    }

    else
    {
        /*
         * PARENT PROCESS
         */
        printf("----- PARENT PROCESS -----\n");
        printf("Parent PID : %d\n", getpid());
        printf("Parent PPID: %d\n", getppid());
        printf("Child PID  : %d\n\n", pid);

        /*
         * Wait before collecting the child.
         *
         * The child finishes after about 35 seconds.
         * The parent waits 45 seconds before waitpid().
         *
         * Therefore, for about 10 seconds the child
         * can appear as Z = Zombie.
         */
        printf("Parent will wait 45 seconds before collecting child.\n");

        sleep(45);

        printf("\nParent is now calling waitpid().\n");

        waitpid(pid, NULL, 0);

        printf("Child has been collected by parent.\n");
        printf("Program completed.\n");
    }

    return 0;
}
```

Execution Output



Observations:

• The program demonstrates the fork() system call creating a child process (PID 734) from the parent (PID 733, PPID 344).

• The original process displays its PID (733) before calling fork(), establishing the parent process identity.

• After fork(), the parent shows 'Parent PID: 733' and child displays 'Child PID: 734' along with its parent's PID (733), confirming the parent-child relationship.

• The program simulates three stages of child execution: Stage 1 (RUNNING/RUNNABLE for 15 seconds), Stage 2 (WAITING/SLEEPING for 20 seconds), and Stage 3 (TERMINATING).

• Parent process uses sleep(45) to wait before calling waitpid(), allowing observation of process states while both parent and child are active.

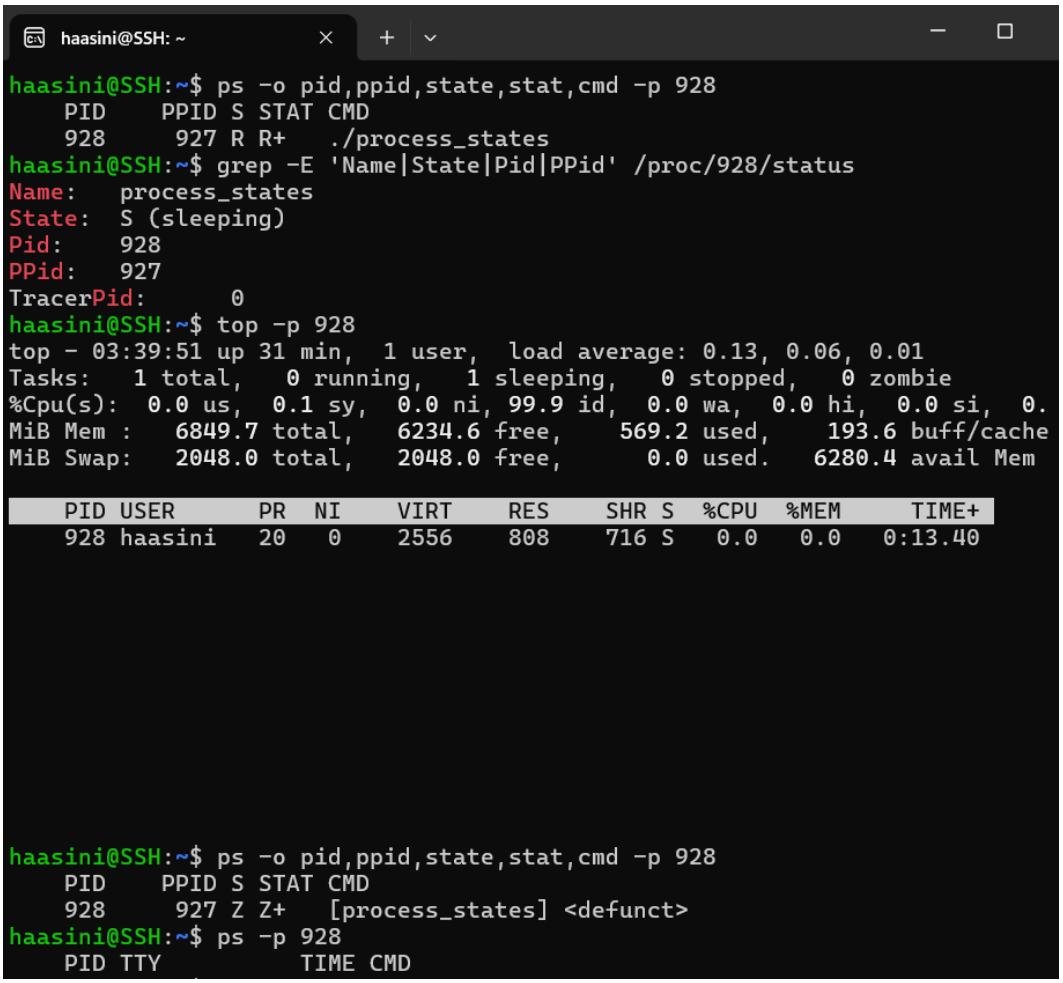
• Program completion message confirms successful waitpid() return, showing proper parent-child synchronization.

Task 2: Process State Transitions Using Linux Monitoring Tools

Question:

Design an experiment to observe process state transitions (Ready, Running, Waiting, Terminated) using Linux monitoring tools such as ps, top, and /proc.

Process State Monitoring



Observations:

• The ps command with options '-o pid,ppid,state,stat,cmd' displays process metadata: PID 928 with PPID 927, state 'R' (running), and status 'R+' (running in foreground).

• The /proc/928/status file shows detailed process information including process state 'S (sleeping)', confirming the process transitioned to sleeping state during execution.

• The top command output shows process 928 using 0.0% CPU and 0.0% memory while sleeping, indicating the process is not consuming CPU time while in the waiting/sleeping state.

• Process state transitions are visible through repeated queries: the process started in 'R' (running) state, moved to 'S' (sleeping) state when sleep() was called, and showed 'Z' (zombie) state after termination during waitpid() collection.

• Memory usage remained constant at 808 bytes RES and 2556 bytes VIRT throughout the state transitions, showing the OS preserves process memory across state changes.

• The monitoring tools demonstrate how the OS abstracts the internal scheduling and state management, allowing user-space programs to observe process behavior without kernel implementation details.